

Dart Task

Application en ligne de commande de gestion de tâches, développée en Dart pur et sans Flutter. Elle permet de créer, consulter, terminer et supprimer des tâches, avec une sauvegarde automatique dans un fichier JSON local.

Fonctionnalités

- Création d'une tâche avec titre, priorité et date limite optionnelle
- Priorités disponibles : `low`, `medium` et `high`
- Liste complète des tâches
- Tri par priorité ou par date limite
- Marquage d'une tâche comme terminée
- Suppression d'une tâche
- Persistance locale dans `tasks.json`
- Validation des données et erreurs métier explicites

Prérequis

- Dart SDK `3.7.2` ou version compatible avec la contrainte du projet
- Un terminal placé à la racine du projet

Vérifier l'installation de Dart :

```
dart --version
```

Installation

Récupérer le projet puis installer les dépendances de développement :

```
git clone <url-du-repository>
cd dart_task
dart pub get
```

Utilisation

Depuis la racine du projet :

```
dart run
```

L'application fonctionne en mode interactif. Les commandes disponibles sont :

Commande	Description
<code>add <titre> [priorité] [YYYY-MM-DD]</code>	Ajoute une tâche. La priorité par défaut est <code>medium</code> .
<code>list</code>	Affiche toutes les tâches dans leur ordre de création.
<code>list priority</code>	Trie les tâches de la priorité la plus haute à la plus basse.
<code>list date</code>	Trie les tâches par date limite, les tâches sans date en dernier.
<code>done <id></code>	Marque la tâche identifiée comme terminée.
<code>delete <id></code>	Supprime la tâche identifiée.
<code>help</code>	Affiche un rappel des commandes principales.
<code>quit</code> ou <code>exit</code>	Ferme l'application.

Exemple de session :

```
> add "Preparer le rapport" high 2026-10-15
Tache ajoutée: 1729000000000000

> add Repondre aux emails low
Tache ajoutée: 1729000000000001

list priority
[ ] 1729000000000000 | high | 2026-10-15 | Preparer le rapport
[ ] 1729000000000001 | low | - | Repondre aux emails

done 1729000000000000
Tache terminee.

delete 1729000000000001
Tache supprimee.
```

Le titre peut contenir plusieurs mots. Lorsqu'une priorité ou une date est fournie, elles doivent être placées à la fin de la commande `add`.

Les données sont sauvegardées dans `tasks.json` à côté du projet. Ce fichier est créé automatiquement au premier ajout et peut être supprimé pour repartir d'une liste vide.

Architecture

Le code est organisé autour de quatre éléments :

- `Task` est une classe abstraite représentant le contrat commun d'une tâche.

- `StandardTask` et `UrgentTask` illustrent l'héritage et la sérialisation du type concret.
- `Repository<T>` est l'interface générique de stockage. `JsonTaskRepository` fournit son implémentation pour un fichier JSON.
- `TaskManager` contient les opérations métier et ne dépend que de `Repository<Task>`, ce qui facilite les tests.

Les erreurs sont signalées avec `InvalidTaskException`, `TaskNotFoundException` et `TaskStorageException`.

Tests

```
dart test
```

La suite couvre notamment :

- l'ajout et la persistance d'une tâche ;
- la conservation des priorités et des dates ;
- le tri par priorité ;
- la terminaison et la suppression ;
- les erreurs de validation et les identifiants inconnus ;
- la restauration d'une `UrgentTask` depuis le JSON.

Pour vérifier également le code avec l'analyseur Dart :

```
dart analyze
```